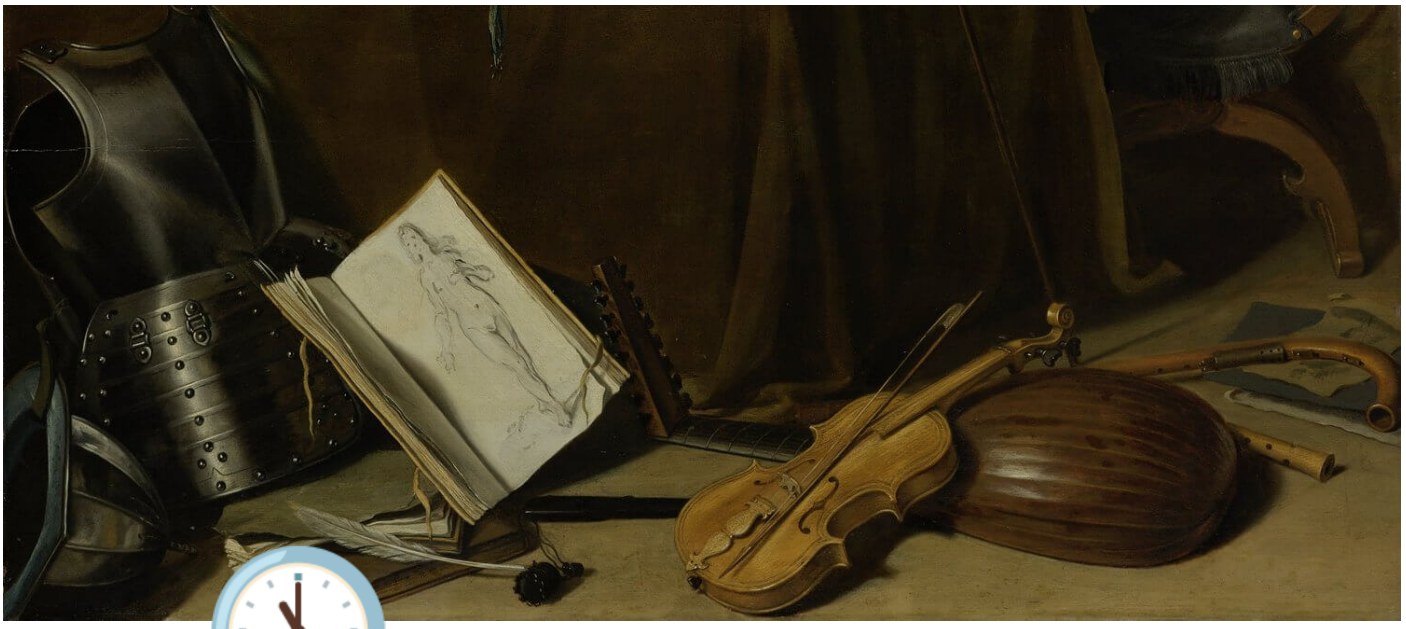


Distributed Food ordering System

Technologies: Apache Kafka · Apache Spark · KRaft · ZeroTier

> 🗒️ Your team's dataset is available here: A2 Dataset Download
> Enter your assigned team number from the team list (e.g., Team 1 → enter `1`).

Your team's dataset is available here: A2 Dataset Download
Enter your assigned team number from the team list (e.g., Team 1 → enter `1`).

1. Introduction

Imagine a food-delivery platform processing a large number of orders. Order data is first published to Kafka, which acts as the platform's distributed messaging backbone. A downstream processing service consumes these orders and uses Spark to clean and enrich them with delivery-related metrics. The rder data is then published back to Kafka as a new topic, where another service consumes it and runs Spark analytics

Made with Super

to understand restaurant and delivery performance.

The assignment is to build this **two-stage, Kafka and Spark-based batch data pipeline** across four VMs.

2. Objective

By completing the assignment, you should be able to:

- configure a **multi-broker Kafka cluster using KRaft**;
- understand how **topics, partitions and replication** work together;
- demonstrate how Kafka maintains **availability of replicated data when a broker fails**;
- build Kafka producers and consumers that move data between independent processing stages;
- use **Apache Spark Batch** to transform raw order data into enriched order-level metrics;
- publish processed data as a new Kafka topic for downstream consumption;
- use a second Spark Batch job to perform **restaurant- and city-level analytics**;

The assignment therefore combines two key ideas:

> **Kafka provides reliable, distributed data movement; Spark provides batch computation over the data at the consumer end.**

3. System Roles

VM	Role	Responsibility
VM1	Producer 1, Spark Job 2, Consumer 2	Publishes `orders`; consumes `order_metrics`; runs Spark Job 2
VM2	KRaft Controller + Kafka Broker 1	Kafka broker and controller
	Kafka Broker 2	Kafka broker and controller

Made with Super

VM4	Consumer 1, Spark Job 1, Producer 2	Consumes `orders`; runs Spark Job 1; publishes `order_metrics`

VM2 and VM3 form the **Kafka broker cluster with VM2 serving as both broker and controller.**

4. Data Flow Overview

The pipeline consists of two batch-processing stages:

Stage 1 - Order Enrichment

1. VM1 publishes the input dataset to Kafka topic `orders`.
2. VM4 consumes `orders` and stores the records on its local filesystem as **`orders_landed.jsonl`**.
3. VM4 runs **Spark Job 1** on `orders_landed.jsonl`.
4. Spark produces the enriched order-level records and writes them to **`order_metrics.jsonl`**.
5. VM4 publishes the records from `order_metrics.jsonl` to Kafka topic `order_metrics`.

Stage 2 - Operations Analytics

1. VM1 consumes `order_metrics` and stores the records on its local filesystem as **`order_metrics_landed.jsonl`**.
2. VM1 runs **Spark Job 2** on `order_metrics_landed.jsonl`.
3. Spark produces the final restaurant and city analytics files.

5. About the Dataset

Each team will receive a unique dataset. All teams follow the same schema and processing rules, but the actual records differ.

Input schema:

```
order_id, customer_id, restaurant_id,
delivery_partner_id, city, order_timestamp,
accepted_timestamp, picked_up_timestamp,
expected_delivery_timestamp, delivered_timestamp,
, order_amount, delivery_fee, status
```

Made with Super

Each record represents a completed food order and contains the

timestamps, participants, location, distance and financial details required for processing.

Cleaning rules

Spark Job 1 must ignore records where:

- the JSON record is malformed;
- a required field is missing or invalid;
- any required timestamp is invalid;
- `distance_km <= 0` ;
- `order_amount <= 0` ;
- `status` is not `DELIVERED` .

For both Spark Jobs 1 and 2, exact duplicate orders must be counted only once.

6. Spark Jobs to be Run

Spark Job 1 - Order Enrichment

Input: `orders_landed.jsonl`

For every valid order, calculate:

Preparation time

```
preparation_time_min =
    picked_up_timestamp - accepted_timestamp
```

Delivery time

```
delivery_time_min =
    delivered_timestamp - picked_up_timestamp
```

Fulfillment time

```
fulfillment_time_min =
    delivered_timestamp - order_timestamp
```

Made with Super

```
delivery_speed_kmph =
    distance_km / (delivery_time_min / 60)
```

Delay

```
delay_min =
    max(0, delivered_timestamp - expected_delivery_timest
```

Delivery status

```
delay_min = 0  → ON_TIME
delay_min > 0  → DELAYED
```

Delay category

```
0 min                → ON_TIME
1–15 min             → SLIGHT_DELAY
16–30 min            → MODERATE_DELAY
>30 min              → SEVERE_DELAY
```

Write the enriched records to:

```
order_metrics.jsonl
```

Expected schema:

`order_id, customer_id, restaurant_id, delivery_partner_id, city, order amount, delivery_fee, distance_km, preparation_time_min, delivery_time_min, fulfillment_time_min, delivery_speed_kmph, delay_min, delivery_status, delay_category`

Then publish the records from this file to Kafka topic:

```
order_metrics
```

Made with Super

Spark Job 2 - Operations Analytics

Input: `order_metrics_landed.jsonl`

Perform exactly these two analytics.

A. Restaurant Performance

Group by `restaurant_id` and calculate:

```
restaurant_id
order_count
avg_order_amount
avg_preparation_time_min
avg_delivery_time_min
delayed_order_count
delay_rate_pct
```

where:

```
delay_rate_pct =
    100 × delayed_order_count / order_count
```

Output:

`restaurant_summary.csv`

B. City Delivery Performance

Group by `city` and calculate:

```
city
order_count
avg_order_amount
avg_delivery_time_min
delayed_order_count
delay_rate_pct
```

Output:

`city_summary.csv`

All calculated numerical values must be rounded to **2 decimal places.**

Made with Super

7. Expected Output

File / Topic	Description
`orders_landed.jsonl`	Records consumed by VM4
`order_metrics.jsonl`	Enriched records produced by Spark Job 1 on VM4; these records must also be published to `order_metrics`
`order_metrics_landed.jsonl`	Records consumed by VM1
`restaurant_summary.csv`	Final restaurant-level analytics
`city_summary.csv`	Final city-level analytics

8. Implementation Notes

- **Do not have Spark consume Kafka directly.** Kafka data must first be consumed into a local file, which is then used as the Spark input.
- **Do not use HDFS.** The intended landing area is the local filesystem.
- The output of **Spark Job 1 must be published to Kafka as `order_metrics`** . Do not transfer the file directly from VM4 to VM1.
- Use **2 partitions and replication factor 2** for both Kafka topics.
- `orders` and `order_metrics` must use the **same Kafka broker cluster**.
- The broker-failure test must be performed by stopping **VM3** verifying that replicated partitions remain available through the surviving brokers (and not VM2, controller handover is out of scope for this assignment).
- Apply the specified cleaning, duplicate-handling, transformations and analytics **exactly as given**; do not add or change business rules.

Made with Super

- Final outputs must be generated from the team's assigned dataset and not hardcoded.

Made with Super